



Algoritmos y Estructuras de Datos
Cátedra Juárez

INFORME DE COMPLEJIDAD ALGORÍTMICA

Nombre del grupo: Grafosaurios

Integrantes:

- Devitt, Francisco Augusto - 105402
- Gonzales, Axel - 114001
- Sciaini, Carola - 111406
- Woiciechowski, Matias - 104837

Resumen:

En este trabajo se desarrollaron e integraron las funcionalidades principales de un juego inspirado en GTA VI, incluyendo un sistema ampliado de vehículos, un concesionario, un garaje con cola de espera, un motor de misiones con mapa variable y un GPS.

El objetivo del informe es evaluar experimentalmente el desempeño de distintos algoritmos de búsqueda de caminos mínimos y seleccionar el más eficiente, en el contexto del juego, para la implementación del sistema de GPS. Los algoritmos elegidos fueron: A*, Dijkstra y Bellman-Ford.

Partimos de la hipótesis de que, para un entorno dinámico y con pesos variables como el mapa del juego, un algoritmo heurístico como A* utilizando una heurística admisible para el tipo de mapa (distancia Manhattan), ofrecería un mejor rendimiento que los otros algoritmos, encontrando los caminos mínimos en menor tiempo.

Los resultados experimentales mostraron que A* cumplió con esta hipótesis, fue consistentemente más eficiente que Dijkstra al expandir una menor cantidad de nodos. Si bien en algunos escenarios límite el comportamiento de ambos algoritmos tendió a aproximarse, en la mayoría de los casos A* logró evitar la exploración de regiones alejadas del destino. Bellman-Ford, en contraste, presentó un costo operacional y temporal significativamente mayor debido a su naturaleza de expansión y relajación exhaustiva.

En conclusión, A* resulta la alternativa más conveniente para el GPS del juego por su balance entre optimalidad y eficiencia práctica.

Introducción

El sistema de GPS del juego debe calcular caminos mínimos entre la posición del jugador y un destino sobre un mapa de una ciudad representado como un grafo ponderado y dirigido. Dado que se recalculan rutas durante el desarrollo de una misión, un algoritmo de búsqueda ineficiente podría generar latencia en el juego, afectando directamente la experiencia del usuario.

En este contexto, no solo interesa que el algoritmo encuentre caminos óptimos, sino que lo haga en tiempos compatibles con un sistema interactivo. Por este motivo, se analizan experimentalmente tres algoritmos de caminos mínimos con enfoques

distintos de exploración, evaluando su comportamiento bajo condiciones controladas y comparables.

Marco teórico mínimo

Expansión de nodos y relajación de aristas

En los algoritmos de caminos mínimos, un nodo expandido es aquel que fue seleccionado debido a que su costo actual es el menor de entre todos los nodos disponibles, por lo que es procesado y se inspeccionan sus aristas salientes. Determina cuántos nodos del grafo se exploran.

La relajación de una arista consiste en evaluar si el camino que pasa por el nodo previamente expandido mejora la distancia conocida hacia uno de sus vecinos. Determina cuántas aristas se evalúan.

Algoritmos comparados:

- **Dijkstra:** Algoritmo que calcula distancias mínimas desde un origen expandiendo iterativamente el nodo con menor costo acumulado. Garantiza la obtención de caminos óptimos en grafos con pesos no negativos, pero al no utilizar información sobre el destino, puede expandir nodos en regiones irrelevantes del grafo antes de encontrar la solución.

La complejidad algorítmica teórica es $O((V+E)\log V)$

- **A*:** Es una extensión de Dijkstra que optimiza la exploración acercándose al destino mediante la incorporación de una función de evaluación $f(n) = g(n) + h(n)$, donde $h(n)$ es la función heurística que estima el costo restante hasta el destino, y $g(n)$ el costo real acumulado.

En grafos tipo grilla, la distancia Manhattan es una heurística admisible, ya que nunca sobreestima el costo real del camino mínimo. Bajo estas condiciones, A* garantiza encontrar un camino óptimo.

La complejidad algorítmica teórica es $O((V+E)\log V)$

En el peor caso, su complejidad asintótica coincide con la de Dijkstra, pero en la práctica reduce significativamente el espacio explorado.

- **Bellman–Ford:** No utiliza una estructura de prioridad para seleccionar nodos prometedores, sino que calcula caminos mínimos mediante la expansión y relajación repetida de la totalidad de nodos y aristas del grafo y, a diferencia

de Dijkstra y A*, puede manejar pesos negativos.
Su complejidad algorítmica teórica es $O(V \cdot E)$.

Hipótesis

- A* emplea una heurística admisible que permite reducir la cantidad de nodos explorados sin perder optimalidad en el camino encontrado.
- Dijkstra, al no utilizar heurística, explora una región más amplia del grafo antes de alcanzar el destino.
- Bellman–Ford presenta un mayor costo operacional debido a su estrategia de exploración exhaustiva de aristas, lo que impacta especialmente en grafos de mayor tamaño.
- En grafos con pesos que distorsionan la distancia geométrica, la eficiencia de A* puede disminuir.

Se espera observar que Dijkstra expanda más nodos que A*, dado que explora el espacio de búsqueda de forma radial desde el origen sin información direccional hacia el objetivo.

En contraste, A* incorpora una heurística basada en la distancia Manhattan que orienta la exploración hacia el destino, reduciendo el número de expansiones y el tiempo de ejecución. Sin embargo, en escenarios de alta complejidad topológica, o con pesos muy variados, se espera que la diferencia relativa entre A* y Dijkstra en número de operaciones y tiempo de ejecución disminuya.

Por último debido a su complejidad $O(V \cdot E)$, se espera que Bellman–Ford presente un crecimiento significativamente mayor en operaciones totales a medida que aumenta el tamaño del grafo

Desarrollo

Para comparar el desempeño de los algoritmos de caminos mínimos considerados para la implementación del GPS del juego, se definió un entorno de prueba controlado y reproducible, así como un conjunto de métricas que permiten evaluar empíricamente su eficiencia

Entorno experimental y reproducibilidad

Los grafos se modelaron como grillas rectangulares, lo que impone restricciones geométricas y topológicas que condicionan el desempeño de los algoritmos de búsqueda. Esta estructura se define mediante los siguientes componentes:

- **Nodos (V):** Cada nodo representa una celda en el mapa, definida unívocamente por coordenadas cartesianas (x,y). Esto permite el uso de heurísticas basadas en distancias geométricas.
- **Aristas (E):** Representan las conexiones entre celdas adyacentes. Son dirigidas y ponderadas. El peso de cada arista no representa únicamente la distancia física, sino un "costo de tránsito" que integra variables como la congestión vehicular o el tipo de terreno.
- **Grado de los Nodos:** Al tratarse de una grilla ortogonal, cada nodo tiene un grado de salida máximo de 4 (conexiones hacia el Norte, Sur, Este y Oeste). Esta característica garantiza que los grafos creados a partir del mapa sean dispersos, cumpliendo la relación aproximada $|E| \approx 2|V|$.

Para modelar las zonas intransitables se omitieron deliberadamente ciertos nodos y aristas a la hora de crear los grafos a partir de un mapa. Esta omisión se realizó manteniendo la conectividad local del grafo y garantizando la existencia de al menos un camino óptimo entre el origen y el destino en cada instancia.

Tanto las variaciones en los costos de aristas como los nodos origen y destino de cada búsqueda se determinaron mediante un generador pseudoaleatorio con semilla fija. Garantizando instancias de pruebas idénticas entre ejecuciones del programa y entre los 3 algoritmos, asegurando condiciones comparables.

Dado que la distancia relativa entre el origen y el destino influyen críticamente en el desempeño, cada algoritmo se evaluó sobre un conjunto de 1000 pares origen/destino por cada mapa. Este volumen de muestras permite obtener un promedio representativo del comportamiento del algoritmo en el mapa general, eliminando el sesgo de casos "afortunados" donde el camino es trivial.

Métricas medidas

Para comparar el desempeño de los algoritmos se utilizaron dos tipos de métricas:

- **Cantidad de operaciones:** Métrica independiente del hardware que aproxima el costo algorítmico intrínseco. Se contabilizaron:
 - Nodos expandidos.
 - Aristas inspeccionadas durante relajaciones.
 - Operaciones sobre cola de prioridad (en A*/Dijkstra)
- **Tiempo de ejecución:** Métrica práctica que refleja el rendimiento observable del GPS. Si bien depende del hardware y del entorno de ejecución, resulta relevante para evaluar la experiencia de usuario.

Las métricas obtenidas se analizaron en función del **tamaño** de las instancias de prueba. Dicho parámetro se definió como la suma de la cantidad de **nodos** $|V|$ y de la cantidad de **aristas** $|E|$ que posee el grafo.

Adicionalmente, se consideraron los siguientes aspectos complementarios para el análisis experimental:

- Mediciones en casos adversos: Se diseñaron escenarios con obstáculos cóncavos y callejones sin salida que contradicen la dirección sugerida por la heurística. El objetivo es medir la degradación del rendimiento de A^* .
- El promedio de los costos de caminos mínimos encontrados: Se utilizó como criterio de validación para confirmar que todos los algoritmos hayan obtenido los mismos caminos mínimos (o en su defecto, del mismo costo), independientemente de sus diferencias en eficiencia temporal y operacional.

Instrumentación

El conteo de operaciones se obtuvo incrementando contadores en los puntos críticos de cada implementación de los algoritmos. La clase especializada *Metricas* es la encargada de llevar el conteo mediante los metodos: `seExpandio()` , `seRelajo()`; `seExtrajo()`; `seInserto()`;

Para el tiempo de ejecución se utilizó `System.nanoTime()` por su precisión a nivel de nanosegundos. Cada algoritmo se ejecutó 10 veces seguidas por cada par origen/destino, se registró el tiempo para cada una y luego se promedió con el propósito de reducir el efecto del hardware sobre esta medición.

Resultados

Se presenta un resumen de las métricas clave para los distintos tamaños de grafos evaluados

Tabla 1: Resultados experimentales promedio

Mapa	Tamaño (V + E)	V	E	Algoritmo	Tiempo prom. (ms)	ops. prom.	Exp.	Rels.	Costo prom.
mapa1.txt	97	31	66	A*	0.0444	46	8	18	5
				Dijkstra	0.0857	85	16	34	5
				Bellman - Ford	0.1406	352	110	242	5
mapa2.txt	215	67	148	A*	0.0341	81	14	32	7
				Dijkstra	0.0475	182	34	75	7
				Bellman - Ford	0.1987	1259	385	873	7
mapa3.txt	460	140	320	A*	0.0405	165	28	68	12
				Dijkstra	0.1021	388	71	165	12
				Bellman - Ford	0.6908	4182	1255	2926	12
mapa4.txt	832	246	586	A*	0.0899	226	38	95	14
				Dijkstra	0.1758	681	123	298	14
				Bellman - Ford	1.7424	10273	2993	7279	14
mapa5.txt	1156	340	816	A*	0.1107	254	42	108	16
				Dijkstra	0.2989	933	168	410	16
				Bellman - Ford	2.4686	15849	4631	11218	16

Tabla 2: Resultados experimentales en casos adversos

Mapa	Tamaño (V + E)	V	E	Algoritmo	ops. prom.	Exp.	Rels.	Costo prom.
complicado1.txt	91	31	60	A*	71	14	26	12
				Dijkstra	79	16	29	12
complicado2.txt	570	190	380	A*	432	86	171	63
				Dijkstra	473	94	188	63
complicado3.txt	1818	550	1268	A*	1158	215	502	79
				Dijkstra	1468	274	639	79
complicado4.txt	2592	844	1748	A*	1366	264	559	78
				Dijkstra	2186	426	897	78

Para visualizar la escalabilidad de los algoritmos, se generaron los siguientes gráficos a partir de los datos:

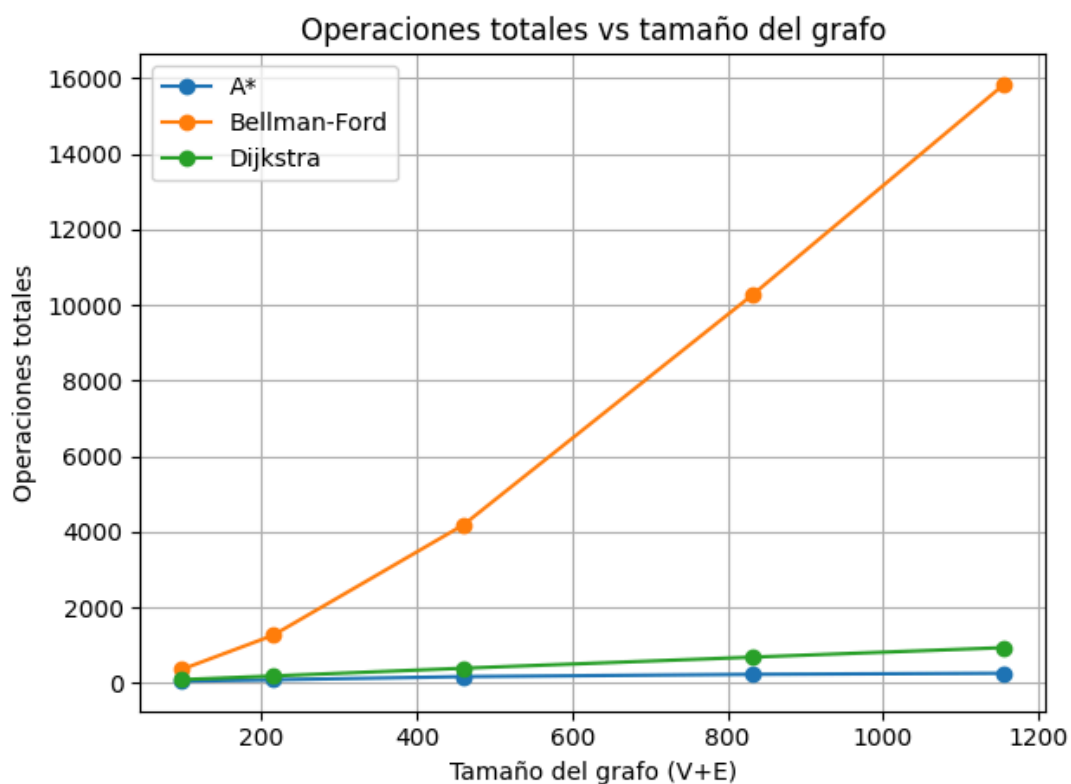


Gráfico 1: Evalúa el esfuerzo algorítmico independiente del hardware.

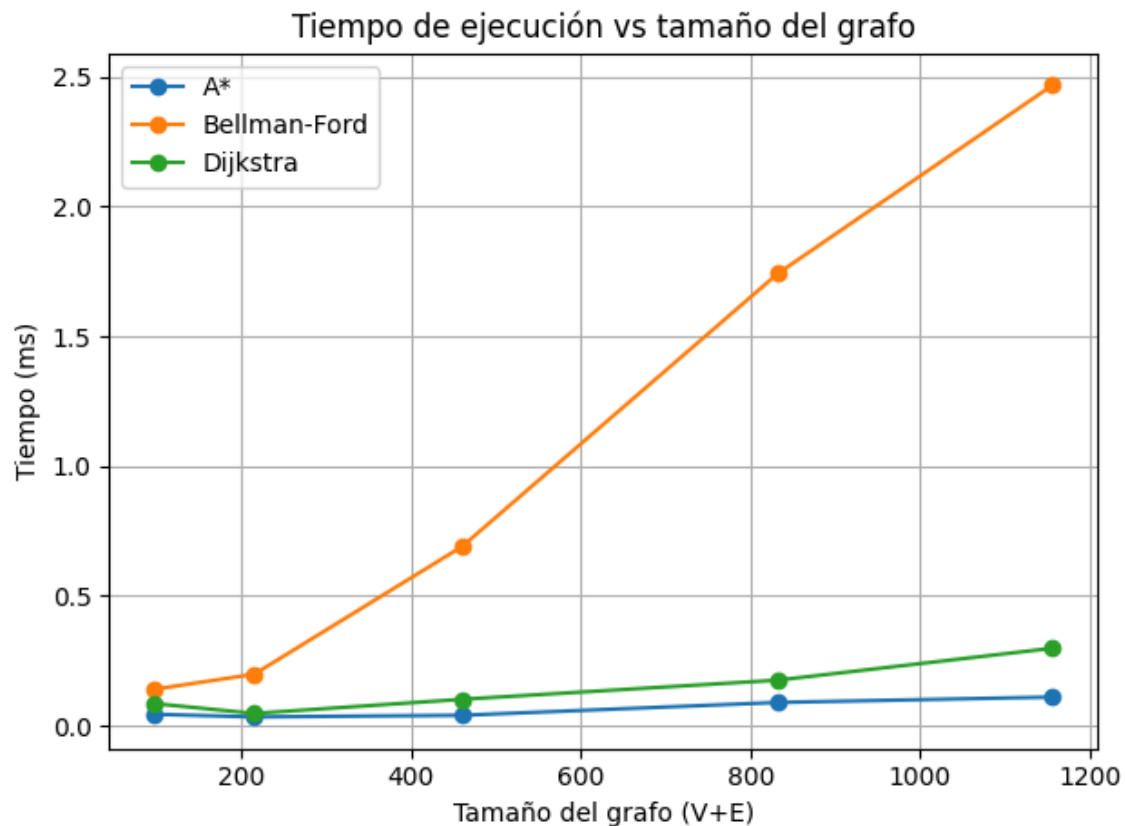


Gráfico 2: Muestra cómo varía la latencia a medida que aumenta la cantidad de nodos y aristas.

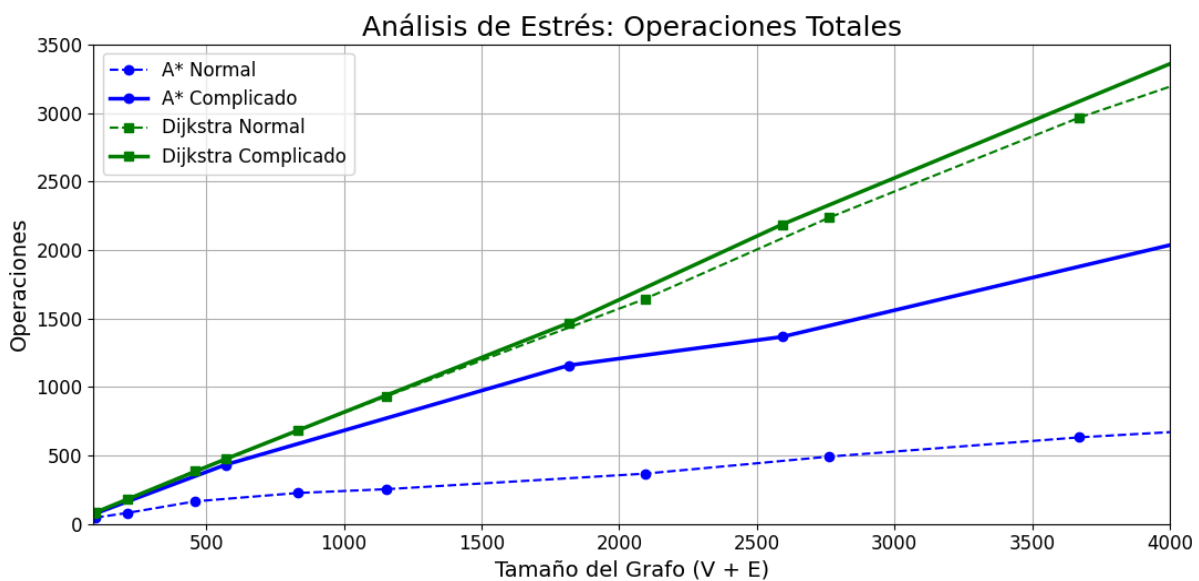


Gráfico 3: Comparación de la eficiencia de A* y Dijkstra en mapas normales versus mapas con obstáculos cóncavos.

Analisis resultados

El análisis de las operaciones totales permite observar el esfuerzo algorítmico real, eliminando variables externas de hardware.

- **A* (Optimización Heurística):** Presenta el desempeño más eficiente en todas las instancias evaluadas. Su curva de crecimiento es notablemente más plana que la de los otros algoritmos, lo que indica que la cantidad de expansiones y relajaciones se mantiene acotada independientemente del tamaño del mapa. Esto valida la eficacia de la función de evaluación $f(n)=g(n)+h(n)$: al priorizar nodos con menor distancia estimada al destino, el algoritmo evita procesar regiones del grafo que no contribuyen al camino óptimo.
- **Dijkstra:** Exhibe una cantidad de operaciones consistentemente superior a A*. Su comportamiento refleja una exploración en "ondas" o círculos concéntricos desde el origen; al carecer de una guía hacia el destino, se ve obligado a relajar una mayor proporción de aristas para garantizar la optimalidad. No obstante, su crecimiento sigue una tendencia log-lineal estable ($O(E \log V)$), consolidándose como un *baseline* robusto y confiable.
- **Bellman-Ford (Relajación Masiva):** La curva de este algoritmo muestra una pendiente pronunciada que escala rápidamente con el tamaño del grafo. Dado que su mecánica se basa en múltiples pasadas sobre la totalidad de las aristas, el volumen de operaciones resulta órdenes de magnitud superior a los métodos basados en colas de prioridad. Este comportamiento lo posiciona como el candidato menos apto para un sistema dinámico como el GPS de un juego, donde el recálculo constante de rutas volvería ineficiente la experiencia de usuario.

Respecto a los mapas con obstáculos cóncavos, se observa que la brecha de rendimiento entre A* y Dijkstra se reduce significativamente, la heurística Manhattan guía a A* hacia el interior del obstáculo y al encontrarse con un callejón sin salida, A* se ve obligado a expandir nodos que inicialmente había descartado, incrementando su conteo de operaciones hasta niveles cercanos a los de Dijkstra. Esto demuestra que la eficiencia de A* es **sensible a la topología del mapa**, Sin embargo niquiera en esos casos llega a ser menos eficiente que una búsqueda ciega.

Comparación y conclusión “de rendimiento”

- **A*** es el mejor en rendimiento (menos tiempo y menos operaciones) y además mantiene calidad del camino (óptimo si la heurística es admisible/consistente).
- **Dijkstra** es correcto y relativamente eficiente, pero al no usar heurística hace más trabajo, por eso queda segundo.
- **Bellman-Ford** queda último: su costo (tiempo y operaciones) crece demasiado, por lo que no conviene para el GPS del juego salvo que hubiera pesos negativos (que en este TP no parecen necesarios).

Aunque los mapas crecen en tamaño, las curvas muestran una diferencia muy clara:

- A* reduce expansión de nodos → menos operaciones → menos tiempo.
- Dijkstra necesita explorar más → más operaciones → más tiempo.
- Bellman-Ford repite relajaciones → muchas operaciones → mucho tiempo.

Conclusiones

A partir de las mediciones realizadas, se concluye que **A*** es la opción más adecuada para el GPS del juego en mapas tipo grilla, ya que obtiene **camino de costo mínimo** con **menor cantidad de expansiones y operaciones** que **Dijkstra**, lo que se traduce en mejores tiempos de ejecución en los casos evaluados. **Dijkstra** funciona como una referencia correcta y eficiente, pero al no utilizar heurística requiere explorar una mayor porción del grafo. Por su parte, **Bellman-Ford** resultó el enfoque más costoso en tiempo y operaciones, por lo que no es recomendable para un GPS que deba ejecutarse repetidas veces durante la partida.